ADVANCED OBJECT ORIENTED PROGRAMMING – (25CS2205E)

Roll No . : 2520030027

Name: Venkata Sai Krishna

SEC-4

PRACTICAL-1:

Aim: To develop a Java-based Bank Account and Money Transfer application that manages bank accounts and performs deposit, withdrawal, fund transfer, freeze, unfreeze, and account closure operations while maintaining account balance and transaction records through validation.

Code:

```
import java.util.*;

class Money {
   long paise;
   Money(double amount) {
      if (amount <= 0) throw new IllegalArgumentException("Invalid amount");
      paise = Math.round(amount * 100);
   }
   Money(long paise) { this.paise = paise; }
   public String toString() { return "₹" + paise / 100.0; }
}

enum AccountStatus { ACTIVE, FROZEN, CLOSED }

class LedgerEntry {
   String type;
   Money amount;
   Date time = new Date();

   LedgerEntry(String type, Money amount) {
      this.type = type;
```

```
        this.amount = amount;
    }

    public String toString() {
        return type + " | " + amount + " | " + time;
    }
}

class BankAccount {
    String number;
    Money balance;
    Money overdraft;
    AccountStatus status = AccountStatus.ACTIVE;
    List<LedgerEntry> ledger = new ArrayList<>();

    BankAccount(String number, double balance, double overdraft) {
        this.number = number;
        this.balance = new Money(Math.round(balance * 100));
        this.overdraft = new Money(Math.round(overdraft * 100));
    }

    void deposit(double amount) {
        check();
        Money m = new Money(amount);
        balance = new Money(balance.paise + m.paise);
        ledger.add(new LedgerEntry("Deposit", m));
    }

    void withdraw(double amount) {
        check();
        Money m = new Money(amount);
        if (balance.paise - m.paise < -overdraft.paise)
            throw new RuntimeException("Overdraft limit exceeded");
        balance = new Money(balance.paise - m.paise);
        ledger.add(new LedgerEntry("Withdrawal", m));
    }
```

```
    void check() {
        if (status != AccountStatus.ACTIVE)
            throw new RuntimeException("Account is not active");
    }

    void freeze() { check(); status = AccountStatus.FROZEN; }
    void unfreeze() {
        if (status != AccountStatus.FROZEN)
            throw new RuntimeException("Account is not frozen");
        status = AccountStatus.ACTIVE;
    }

    void close() {
        check();
        if (balance.paise != 0)
            throw new RuntimeException("Balance must be zero");
        status = AccountStatus.CLOSED;
    }

    void show() {
        System.out.println(number + " | " + balance + " | " + status);
    }

    void showLedger() {
        ledger.forEach(System.out::println);
    }
}

class FundTransfer {
    static void transfer(BankAccount from, BankAccount to, double amount) {
        from.check();
        to.check();
        Money m = new Money(amount);

        if (from.balance.paise - m.paise < -from.overdraft.paise)
```

```
            throw new RuntimeException("Insufficient balance");

        long oldFrom = from.balance.paise;
        long oldTo = to.balance.paise;
        int oldFromLedger = from.ledger.size();
        int oldToLedger = to.ledger.size();

        try {
            from.withdraw(amount);
            to.deposit(amount);
            from.ledger.set(from.ledger.size() - 1,
                    new LedgerEntry("Transfer Sent", m));
            to.ledger.set(to.ledger.size() - 1,
                    new LedgerEntry("Transfer Received", m));
        } catch (RuntimeException e) {
            from.balance = new Money(oldFrom);
            to.balance = new Money(oldTo);
            from.ledger.subList(oldFromLedger, from.ledger.size()).clear();
            to.ledger.subList(oldToLedger, to.ledger.size()).clear();
            throw e;
        }
    }
}

public class practical1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Map<String, BankAccount> accounts = new HashMap<>();

        while (true) {
            System.out.println("\n1.Create 2.Deposit 3.Withdraw 4.Transfer");
            System.out.println("5.Freeze 6.Unfreeze 7.Close 8.Details 9.Ledger 10.Exit");
            System.out.print("Choice: ");
            int ch = sc.nextInt();

            try {
```

```
if (ch == 1) {
   System.out.print("Account number: ");
   String n = sc.next();
   System.out.print("Balance and overdraft: ");
   accounts.put(n, new BankAccount(n, sc.nextDouble(), sc.nextDouble()));
} else if (ch == 10) {
   break;
} else {
   System.out.print("Account number: ");
   BankAccount a = accounts.get(sc.next());

   if (a == null) throw new RuntimeException("Account not found");

   switch (ch) {
      case 2:
         System.out.print("Amount: ");
         a.deposit(sc.nextDouble());
         break;
      case 3:
         System.out.print("Amount: ");
         a.withdraw(sc.nextDouble());
         break;
      case 4:
         System.out.print("Destination account: ");
         BankAccount b = accounts.get(sc.next());
         if (b == null) throw new RuntimeException("Account not found");
         System.out.print("Amount: ");
         FundTransfer.transfer(a, b, sc.nextDouble());
         break;
      case 5: a.freeze(); break;
      case 6: a.unfreeze(); break;
      case 7: a.close(); break;
      case 8: a.show(); break;
      case 9: a.showLedger(); break;
      default: System.out.println("Invalid choice");
   }
```

```
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
    sc.close();
  }
}
```

Output:

```
PS C:\VS Code\src\AOOP> javac practical1.java
>> java practical1

1.Create 2.Deposit 3.Withdraw 4.Transfer
5.Freeze 6.Unfreeze 7.Close 8.Details 9.Ledger 10.Exit
Choice: 1
Account number: 12345678
Balance and overdraft: 9000
100

1.Create 2.Deposit 3.Withdraw 4.Transfer
5.Freeze 6.Unfreeze 7.Close 8.Details 9.Ledger 10.Exit
Choice: 8
Account number: 12345678
12345678 | ?9000.0 | ACTIVE

1.Create 2.Deposit 3.Withdraw 4.Transfer
5.Freeze 6.Unfreeze 7.Close 8.Details 9.Ledger 10.Exit
Choice: 2
Account number: 12345678
Amount: 9000

1.Create 2.Deposit 3.Withdraw 4.Transfer
5.Freeze 6.Unfreeze 7.Close 8.Details 9.Ledger 10.Exit
Choice: 8
Account number: 12345678
12345678 | ?18000.0 | ACTIVE
```

Results:

1. Successfully created and managed bank accounts.
2. Implemented deposit and withdrawal operations with amount validation.

3. Implemented fund transfers between two bank accounts.
4. Prevented withdrawals beyond the permitted overdraft limit.
5. Restricted transactions on frozen and closed accounts.
6. Implemented account freezing, unfreezing, and closing operations.
7. Maintained a transaction ledger to record deposits, withdrawals, and transfers.
8. Implemented rollback handling to restore account balances if a transfer fails.

Conclusion: The Bank Account and Money Transfer application was successfully implemented in Java. It demonstrates object-oriented programming, encapsulation, validation, transaction management, and ledger maintenance.